

Taller autoguiado

Introducción práctica a gRPC con Python

Sistemas Distribuidos — duración estimada: 2 horas — trabajo individual

Cómo usar esta guía

Este es un taller autocontenido: sigue las secciones en orden, ejecuta cada comando en tu propia máquina (Windows o Linux) y verifica los resultados esperados antes de continuar. No necesitas conocimientos previos de gRPC; sí se asume manejo básico de Python y de la línea de comandos.

1. Objetivos de aprendizaje

Al finalizar este taller, el estudiante estará en capacidad de:

- Explicar qué es gRPC, qué problema resuelve y en qué se diferencia de una API REST tradicional.
- Describir el papel de Protocol Buffers (protobuf) como lenguaje de definición de interfaces (IDL) y formato de serialización.
- Definir un servicio gRPC simple en un archivo .proto.
- Generar código Python cliente/servidor a partir de un archivo .proto usando protoc.
- Implementar y ejecutar un servidor gRPC y un cliente que lo consuma, en Windows o en Linux.
- Identificar los cuatro tipos de llamada de gRPC (unaria, streaming del servidor, streaming del cliente y bidireccional) y reconocer un ejemplo de cada una.

2. ¿Qué es gRPC?

gRPC ("gRPC Remote Procedure Calls") es un framework de código abierto, creado originalmente por Google, para invocar funciones que se ejecutan en otra máquina como si fueran locales (Remote Procedure Call o RPC). En lugar de pensar en "recursos" y verbos HTTP como en REST, en gRPC se piensa en términos de servicios con métodos, de forma muy similar a llamar una función en un lenguaje de programación.

Tres decisiones de diseño explican por qué gRPC es tan usado en sistemas distribuidos y arquitecturas de microservicios:

1. HTTP/2 como transporte: permite multiplexar muchas llamadas sobre una sola conexión TCP, cabeceras comprimidas y streaming bidireccional nativo.
2. Protocol Buffers como IDL y formato binario: el contrato del servicio (métodos, tipos de los mensajes) se define una sola vez en un archivo .proto, y a partir de él se genera

automáticamente el código de cliente y servidor en el lenguaje que se necesite (Python, Java, Go, C++, entre otros).

3. Cuatro patrones de comunicación integrados: unaria (petición-respuesta clásica), streaming del servidor, streaming del cliente y streaming bidireccional, todos definidos declarativamente en el .proto.

gRPC vs. REST, en una frase

REST expone recursos identificados por URLs y típicamente usa JSON en texto plano; gRPC expone métodos de un servicio, serializa en binario con protobuf y aprovecha HTTP/2, lo que suele traducirse en mensajes más pequeños y menor latencia, a costa de ser menos "navegable" desde un navegador o con curl.

3. Prerrequisitos

Necesitas lo siguiente antes de comenzar (ambos sistemas operativos):

- Python 3.8 o superior instalado.
- Acceso a una terminal (PowerShell o CMD en Windows; una shell tipo bash en Linux).
- Un editor de texto o IDE (VS Code, PyCharm, o el que prefieras).
- Conexión a internet para instalar paquetes con pip.

Verifica tu versión de Python antes de continuar:

```
python --version
# En algunas distribuciones de Linux puede ser necesario usar:
python3 --version
```

4. Estructura y tiempos sugeridos

El taller está diseñado para completarse en aproximadamente 2 horas, trabajando a un ritmo tranquilo. Sugerencia de distribución del tiempo:

Sección	Tiempo	Qué obtienes al final
5. Entorno de trabajo	15 min	Entorno virtual con grpcio y grpcio-tools instalados
6. Definir el .proto	15 min	Contrato del servicio Greeter definido
7. Generar código	10 min	Stubs de Python generados a partir del .proto
8. Servidor	25 min	Servidor gRPC unario funcionando
9. Cliente	20 min	Cliente que consume el servicio y muestra la

Sección	Tiempo	Qué obtienes al final
		respuesta
10. Streaming del servidor	20 min	Segundo método que devuelve varios mensajes
11. Errores comunes / 12. Ejercicios	15 min	Depuración autónoma y refuerzo de conceptos

5. Preparar el entorno de trabajo

Crea una carpeta para el taller y, dentro de ella, un entorno virtual de Python. Usa la columna que corresponda a tu sistema operativo.

5.1 Crear la carpeta y el entorno virtual

Paso	Linux	Windows
1. Crear carpeta	<code>mkdir taller-grpc</code> <code>cd taller-grpc</code>	<code>mkdir taller-grpc</code> <code>cd taller-grpc</code>
2. Crear entorno virtual	<code>python3 -m venv venv</code>	<code>python -m venv venv</code>
3. Activar entorno	<code>source venv/bin/activate</code>	<code>venv\Scripts\activate</code>

Si la activación en Windows PowerShell falla con un error de política de ejecución de scripts, ejecuta una sola vez (como el usuario actual, no como administrador):

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

En ambos casos, sabrás que el entorno está activo porque el prompt de la terminal mostrará el prefijo (venv) al inicio de la línea.

5.2 Instalar las librerías de gRPC

Con el entorno virtual activo (en cualquiera de los dos sistemas operativos), instala:

```
pip install grpcio grpcio-tools
```

grpcio contiene el runtime de gRPC (lo que usan cliente y servidor en tiempo de ejecución); grpcio-tools incluye el compilador protoc con soporte para generar código Python.

Punto de control

Ejecuta `python -c "import grpc; print(grpc.__version__)"` y confirma que se imprime un número de versión sin errores antes de continuar. Si ves un error de módulo no encontrado, revisa que el entorno virtual esté activo.

6. Definir el contrato del servicio (.proto)

Crea un archivo llamado greeter.proto en la carpeta del taller, con el siguiente contenido:

```
syntax = "proto3";

package greeter;

// Servicio de saludo
service Greeter {
  // Llamada unaria: una petición, una respuesta
  rpc SayHello (HelloRequest) returns (HelloReply);

  // Streaming del servidor: una petición, varias respuestas
  rpc SayHelloRepeatedly (HelloRequest) returns (stream HelloReply);
}

message HelloRequest {
  string name = 1;
}

message HelloReply {
  string message = 1;
}
```

Observa la estructura del archivo:

- service declara el servicio y sus métodos (los "procedimientos remotos").
- message define la forma de cada mensaje intercambiado, como si fueran clases con campos tipados.
- El número después de cada campo (name = 1) es la posición del campo dentro del mensaje binario serializado, no un valor por defecto; debe ser único dentro del mensaje.
- La palabra clave stream antes del tipo de retorno indica que ese método puede enviar varias respuestas para una sola petición (streaming del servidor).

7. Generar el código Python a partir del .proto

Con el entorno virtual activo y ubicado en la carpeta del taller (donde está greeter.proto), ejecuta:

```
python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. greeter.proto
```

Este comando funciona igual en Windows y en Linux, porque se invoca a través del módulo de Python instalado en el entorno virtual. El resultado son dos archivos nuevos en la carpeta:

- greeter_pb2.py: contiene las clases Python de los mensajes (HelloRequest, HelloReply), generadas a partir de los message del .proto.
- greeter_pb2_grpc.py: contiene el stub del cliente (GreeterStub) y la clase base del servidor (GreeterServicer) para el servicio Greeter.

Importante

No edites manualmente greeter_pb2.py ni greeter_pb2_grpc.py: se regeneran cada vez que cambias el .proto y cualquier edición manual se perderá. Todo el código que tú escribes va en archivos separados (server.py y client.py).

8. Implementar el servidor

Crea un archivo `server.py` con el siguiente contenido:

```
import time
from concurrent import futures

import grpc
import greeter_pb2
import greeter_pb2_grpc

class GreeterServicer(greeter_pb2_grpc.GreeterServicer):
    def SayHello(self, request, context):
        # 'request' es una instancia de HelloRequest
        # Debemos devolver una instancia de HelloReply
        return greeter_pb2.HelloReply(
            message=f'Hola, {request.name}!'
        )

    def SayHelloRepeatedly(self, request, context):
        # Método de streaming: usamos 'yield' en vez de 'return'
        for i in range(5):
            yield greeter_pb2.HelloReply(
                message=f'Hola #{i + 1}, {request.name}!'
            )
            time.sleep(1)

def serve():
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    greeter_pb2_grpc.add_GreeterServicer_to_server(
        GreeterServicer(), server
    )
    server.add_insecure_port('[::]:50051')
    server.start()
    print('Servidor gRPC escuchando en el puerto 50051...')
    server.wait_for_termination()

if __name__ == '__main__':
    serve()
```

Puntos clave del servidor:

- `GreeterServicer` hereda de la clase base generada e implementa cada método del `.proto` como un método de Python.
- `SayHello` es una llamada unaria: recibe un `HelloRequest` y hace return de un único `HelloReply`.
- `SayHelloRepeatedly` es streaming del servidor: en vez de return usa `yield` varias veces, enviando cada `HelloReply` tan pronto se genera.
- `add_insecure_port('[::]:50051')` abre el puerto 50051 sin cifrado TLS; es válido para este taller local, pero en producción se usaría `add_secure_port` con credenciales.

Ejecuta el servidor:

Paso	Linux	Windows
Iniciar servidor	<code>python server.py</code>	<code>python server.py</code>

Deja esta terminal abierta (el servidor queda corriendo en primer plano); usarás una segunda terminal para el cliente.

9. Implementar el cliente

Abre una segunda terminal, actívale el mismo entorno virtual (repite el paso 3 de la sección 5.1 en esa nueva terminal) y crea un archivo `client.py`:

```
import grpc
import greeter_pb2
import greeter_pb2_grpc

def run():
    # 'with' cierra el canal automáticamente al terminar
    with grpc.insecure_channel('localhost:50051') as channel:
        stub = greeter_pb2_grpc.GreeterStub(channel)

        # --- Llamada unaria ---
        response = stub.SayHello(
            greeter_pb2.HelloRequest(name='Estudiante')
        )
        print('Respuesta unaria:', response.message)

        # --- Llamada con streaming del servidor ---
        print('Respuestas en streaming:')
        for response in stub.SayHelloRepeatedly(
            greeter_pb2.HelloRequest(name='Estudiante')
        ):
            print(' ->', response.message)

if __name__ == '__main__':
    run()
```

Con el servidor corriendo en la primera terminal, ejecuta el cliente en la segunda:

Paso	Linux	Windows
Ejecutar cliente	<code>python client.py</code>	<code>python client.py</code>
Salida esperada Deberías ver primero la línea de la respuesta unaria y luego cinco líneas de streaming, una por segundo, correspondientes a los cinco mensajes que el servidor produce con <code>yield</code> .		

10. Los cuatro tipos de llamada gRPC (referencia)

Ya implementaste dos de los cuatro tipos de llamada que soporta gRPC. La siguiente tabla resume los cuatro, para que los reconozcas en otros proyectos:

Tipo	Firma en el .proto	Ejemplo de uso típico
Unaria	<code>rpc M (Req) returns (Res);</code>	Consultar un dato puntual (ya implementada: SayHello)
Streaming del servidor	<code>rpc M (Req) returns (stream Res);</code>	Recibir eventos o notificaciones continuas (ya implementada: SayHelloRepeatedly)
Streaming del cliente	<code>rpc M (stream Req) returns (Res);</code>	Subir muchos datos y recibir un solo resumen (ej. subir un archivo por partes)
Bidireccional	<code>rpc M (stream Req) returns (stream Res);</code>	Chat en tiempo real, sincronización continua entre dos partes

Los otros dos tipos (streaming del cliente y bidireccional) quedan propuestos como ejercicio opcional en la sección 12; no son obligatorios para este taller.

11. Errores comunes y cómo resolverlos

Síntoma	Causa probable y solución
ModuleNotFoundError: No module named 'grpc'	El entorno virtual no está activo, o pip install se ejecutó fuera de él. Verifica el prefijo (venv) en el prompt y repite la instalación.
ModuleNotFoundError: No module named 'greeter_pb2'	No se generó el código a partir del .proto, o el comando protoc se ejecutó desde otra carpeta. Repite el paso 7 desde la carpeta que contiene greeter.proto.
StatusCode.UNAVAILABLE al ejecutar el cliente	El servidor no está corriendo, se cerró la terminal del servidor, o se está usando un puerto distinto a 50051 en cliente y servidor. Verifica que server.py siga en ejecución.
El script protoc no reconoce '-m grpc_tools.protoc'	grpcio-tools no quedó instalado en el entorno virtual activo. Ejecuta de nuevo pip install grpcio-tools con el entorno activado.
En Windows, 'activate' no se ejecuta ('no se reconoce como...')	Se está usando el ejecutable equivocado para la shell. En PowerShell usa venv\Scripts\Activate.ps1; en CMD usa venv\Scripts\activate.bat.

12. Ejercicios de refuerzo

Completa al menos los ejercicios 1 y 2 antes de dar por terminado el taller. Los ejercicios 3 y 4 son opcionales, para quienes terminen antes de tiempo.

Ejercicio 1 (obligatorio) — Nuevo campo en el mensaje

Agrega un campo entero age a HelloRequest en el .proto (recuerda usar un número de campo nuevo, por ejemplo 2). Regenera el código (paso 7), y modifica SayHello para que el mensaje de respuesta incluya la edad recibida, por ejemplo: "Hola, Estudiante! Tienes 20 años."

Ejercicio 2 (obligatorio) — Nuevo método unario

Agrega al servicio Greeter un método unario SayGoodbye(HelloRequest) returns (HelloReply) que devuelva un mensaje de despedida. Impleméntalo en el servidor e invócalo desde el cliente.

Ejercicio 3 (opcional) — Manejo de errores

Modifica SayHello para que, si name llega vacío, el servidor devuelva un error usando context.set_code(grpc.StatusCode.INVALID_ARGUMENT) y context.set_details(...) en lugar de un HelloReply. Captura ese error en el cliente con un bloque try/except grpc.RpcError.

Ejercicio 4 (opcional) — Streaming del cliente

Investiga la sintaxis rpc M (stream Req) returns (Res) y agrega al servicio un método SumAges(stream HelloRequest) returns (HelloReply) que reciba varios HelloRequest (cada uno con un campo age) y devuelva un único HelloReply con la suma total. Esto requiere iterar sobre request_iterator en el servidor y, en el cliente, pasar un generador de mensajes en vez de un único mensaje.

Para seguir profundizando

La documentación oficial (grpc.io/docs/languages/python) incluye ejemplos completos de los cuatro tipos de llamada, autenticación con TLS y manejo avanzado de errores; es una buena siguiente lectura una vez completado este taller.